



Institución
Universitaria
Reacreditada en Alta Calidad

ESTRUCTURA DE DATOS Y »»» LABORATORIO «««

580506004-8

William Giraldo Escobar
Facultad de Ingenierías
Departamento de Sistemas de Información



Estructura de datos con memoria dinámica

Las estructuras de datos con memoria dinámica se usan para **optimizar** el uso de **memoria** cuando **almacenamos** datos

Estas estructuras siguen siendo **colecciones** de datos, pero con formas de acceso **diferentes**

Las estructuras de datos de memoria dinámica son:

- Listas enlazadas simples
- Listas enlazadas dobles
- Listas circulares
- *Pilas*
- Colas
- Árboles

COLAS (queues)

Es un tipo de estructura utilizada para *almacenar* datos, y acceder a ellos en un *orden* específico.

Las colas son una *colección* ordenada de elementos que se asemejan a una 'fila', donde el primer elemento en ser almacenado es el primer elemento en ser eliminado de la misma.



COLAS (queues)

- Es una estructura de datos en donde el primer elemento en entrar es el primero en salir (FIFO: First In – First Out)
- Cada que agregamos (enqueue) un nuevo elemento, se agrega al final de la cola
- Los elementos se eliminan (dequeue) en estricto orden a partir del primero

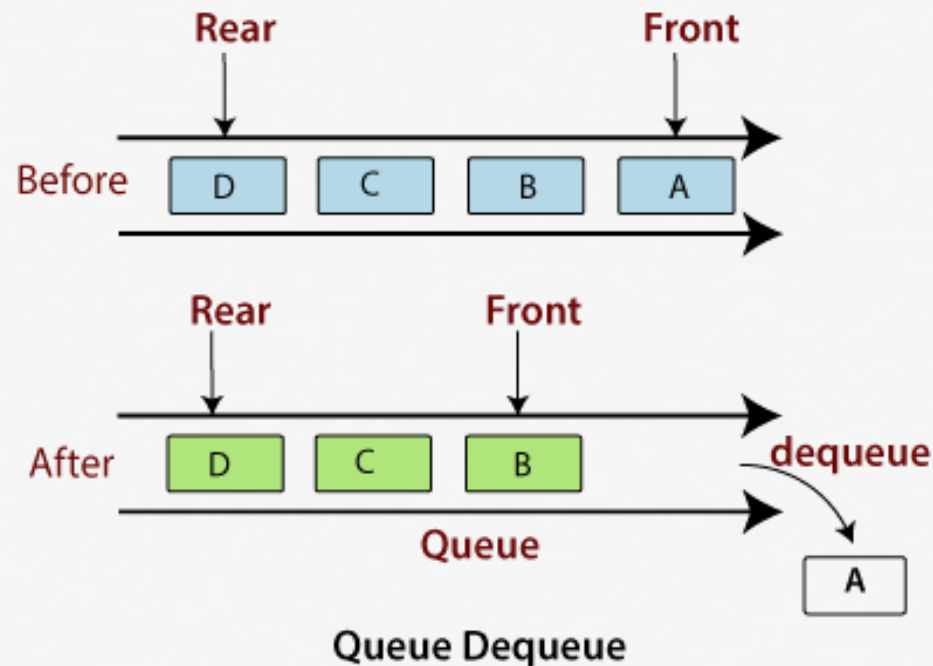


COLAS (queues)

En Python Podemos definir colas bajo a partir de las siguientes estructuras:

- Listas
- Listas enlazadas simples
- Listas doblemente enlazadas

Al igual que en las pilas, podemos programar una cola de tamaño definido, o una cola de tamaño dinámico

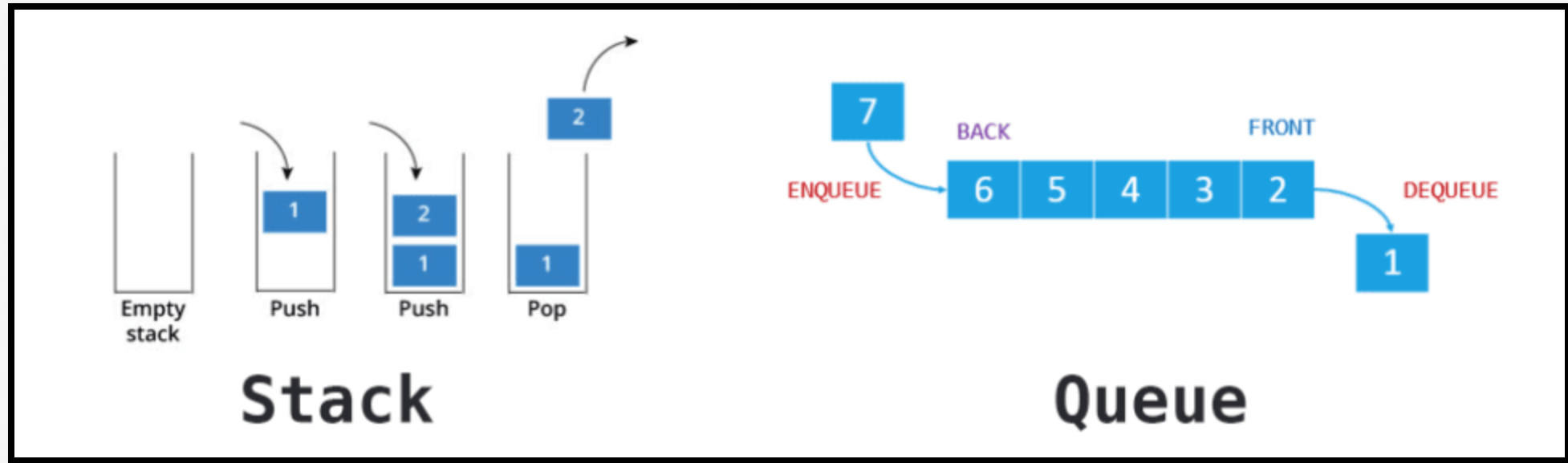


COLAS (queues)

En las pilas las *dos únicas* operaciones posibles son *insertar* y *eliminar*, conocidas también como *enqueue* y *dequeue*.

<i>CrearCola</i>	Inicia la cola como vacía.
<i>Insertar</i>	Añade un elemento por el final de la cola.
<i>Quitar</i>	Retira (extrae) el elemento frente de la cola.
<i>Cola vacía</i>	Comprobar si la cola no tiene elementos.
<i>Cola llena</i>	Comprobar si la cola está llena de elementos.
<i>Frente</i>	Obtiene el elemento frente o primero de la cola.
<i>Tamaño de la cola</i>	Número de elementos máximo que puede contener la cola.

PILAS Y COLAS DIFERENCIAS



COLAS (queues)

Definimos la clase queue (cola)

```
class Queue:
    def __init__(self, max_size=None):
        self.items = []
        self.max_size = max_size
```


COLAS (queues)

Definimos la clase queue (cola)

```
class Queue:
    def __init__(self, max_size=None):
        self.items = []
        self.max_size = max_size
```

```
q = Queue(3)
```

COLAS (queues)

El método `is_empty` retorna `True` si la cola está vacía y `False` en caso contrario

```
def is_empty(self):  
    return len(self.items) == 0
```

COLAS (queues)

El método `is_full` retorna `True` si la cola está completamente llena, y `False` en caso contrario

```
def is_full(self):  
    if self.max_size is not None:  
        return len(self.items) >= self.max_size  
    return False
```

COLAS (queues)

Recordemos que el método enqueue lo usamos para agregar nuevos elementos al final de la cola

```
def enqueue(self, item):  
    if not self.is_full():  
        self.items.append(item)  
    else:  
        print("La cola está llena. No se puede agregar el elemento:", item)
```

Preguntamos si la cola ya está
llena antes de agregar un
nuevo elemento

COLAS (queues)

Recordemos que el método enqueue lo usamos para agregar nuevos elementos al final de la cola

```
def enqueue(self, item):  
    if not self.is_full():  
        self.items.append(item)  
    else:  
        print("La cola está llena. No se puede agregar el elemento:", item)
```

```
q.enqueue(1)  
q.enqueue(2)  
q.enqueue(3)
```


COLAS (queues)

Recordemos que el método dequeue lo usamos para eliminar los elementos de la cola

```
def dequeue(self):  
    if not self.is_empty():  
        return self.items.pop(0)  
    else:  
        print("La cola está vacía")  
        return None
```

COLAS (queues)

Recordemos que el método dequeue lo usamos para eliminar los elementos de la cola

```
q.enqueue(1)  
q.enqueue(2)  
q.enqueue(3)  
q.dequeue()
```

COLAS (queues)

El método size imprime la cantidad de elementos almacenados en la cola

```
def size(self):  
    return len(self.items)
```

COLAS (queues)

El método size imprime la cantidad de elementos almacenados en la cola

```
def size(self):  
    return len(self.items)
```

```
print("Tamaño de la cola:", q.size())
```

COLAS (queues)

El método `print_front` lo usamos para imprimir el primer elemento de la cola (que será el siguiente en ser eliminado)

```
def print_front(self):  
    if not self.is_empty():  
        print("Primer elemento de la cola:", self.items[0])  
    else:  
        print("La cola está vacía")
```

COLAS (queues)

El método `print_front` lo usamos para imprimir el primer elemento de la cola (que será el siguiente en ser eliminado)

```
def print_front(self):  
    if not self.is_empty():  
        print("Primer elemento de la cola:", self.items[0])  
    else:  
        print("La cola está vacía")
```

```
q.print_front()
```




ACTIVIDAD

- Añada el método `print_all` que imprima todos los elementos de la cola
- Defina una nueva cola, pero esta vez defínala de tamaño dinámico
Comience eliminando el atributo `max_size`



Institución
Universitaria
Reacreditada en Alta Calidad

¡MUCHAS GRACIAS!

A decorative graphic consisting of several horizontal lines in various colors (yellow, orange, red, purple, blue, green) and a stylized white 'C' shape on the right side.